Отчёт по лабораторной работе №5

Имитационное моделирование: аппарат сетей Петри

Глущенко Евгений Игоревич

2026-04-17

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**
3.1 Сети Петри 7
3.2 Задача обедающих философов 7
3.3 Стохастическое моделирование 8

4 Выполнение лабораторной работы **9**
4.1 Подготовка окружения 9
4.2 Структура проекта 10
4.3 Описание программной модели 10
4.4 Базовый эксперимент 11
4.5 Результаты классической сети 12
4.6 Результаты сети с арбитром 14
4.7 Анимация процесса 16
4.8 Итоговый сравнительный график 16
4.9 Параметрическое исследование 17
4.10 Генерация literate-форматов 19

5 Анализ результатов **21**

6 Выводы **22**

Список иллюстраций

4.1 Переход в каталог проекта лабораторной работы 9
4.2 Проверка версии Julia 9
4.3 Подготовка окружения Julia-проекта 9
4.4 Запуск базового эксперимента 11
4.5 Эволюция маркировки классической сети 13
4.6 Эволюция маркировки сети с арбитром 15
4.7 Запуск сценария построения анимации 16
4.8 Запуск сценария итогового сравнительного графика . . . 16
4.9 Сравнение состояний `Eat_i` для двух сетей 17
4.10 Запуск параметрического исследования 18
4.11 Частота возникновения deadlock в параметрическом исследовании 18
4.12 Генерация clean, notebook и Quarto-версий скриптов . . 19
4.13 Итоговая структура созданных файлов 20

Список таблиц

1 Цель работы

Изучить аппарат сетей Петри на примере задачи обедающих философов, реализовать классическую и модифицированную модель на языке Julia, выполнить вычислительные эксперименты, получить графики и таблицы результатов, подготовить literate-версии скриптов и собрать итоговую документацию по лабораторной работе.

Исполнитель работы: Глущенко Евгений Игоревич.
Группа: НФИбд-01-23.
Студенческий билет: 1132239110.

2 Задание

В рамках лабораторной работы требовалось:

1. Создать рабочий каталог и проект Julia в структуре `DrWatson`.
2. Установить необходимые пакеты для моделирования, визуализации и документирования.
3. Реализовать модель сети Петри для задачи обедающих философов.
4. Выполнить базовый эксперимент для классической сети и сети с арбитром.
5. Построить анимацию изменения маркировки.
6. Сформировать итоговый сравнительный график по состояниям `Eat_i`.
7. Выполнить параметрическое исследование по `N`, `tmax` и `seed`.
8. Подготовить literate-версии скриптов и производные форматы `clean`, `md`, `ipynb`.
9. Описать все полученные графики, CSV-таблицы и листинги кода.

3 Теоретическое введение

3.1 Сети Петри

Сеть Петри представляет собой двудольный ориентированный граф, в котором вершины разделены на два типа: позиции и переходы. Позиции описывают состояния системы или наличие ресурсов, переходы задают события, а дуги определяют условия срабатывания переходов. Текущее состояние сети называется маркировкой и задаётся числом фишек в каждой позиции [1].

Формально сеть Петри можно записать как

$$N = (P, T, F, M_0),$$

где `P` – множество позиций, `T` – множество переходов, `F` – множество дуг, а `M_0` – начальная маркировка.

Переход считается разрешённым, если во всех его входных позициях есть нужное число фишек. При срабатывании переход удаляет фишки из входных позиций и добавляет фишки в выходные позиции.

3.2 Задача обедающих философов

В задаче обедающих философов `N` философов сидят за круглым столом. Между соседними философами находится по одной вилке.

Чтобы начать есть, философ должен получить две вилки: левую и правую. Если все философы одновременно возьмут по одной вилке, система перейдёт в состояние взаимной блокировки: каждый ждёт вторую вилку, но ни одна вилка не свободна.

В модели использованы следующие позиции:

- `Think_i` – философ `i` размышляет;
- `Hungry_i` – философ `i` взял левую вилку и ожидает правую;
- `Eat_i` – философ `i` ест;
- `Fork_i` – вилка `i` свободна.

В модифицированной сети добавляется позиция `Arbiter` с `N - 1` фишками. Она не позволяет всем философам одновременно войти в стадию захвата ресурсов и тем самым устраняет тупиковую конфигурацию.

3.3 Стохастическое моделирование

Для имитационного моделирования используется событийная схема, близкая к алгоритму Гиллеспи [2]. На каждом шаге вычисляются интенсивности допустимых переходов, случайно выбирается следующий переход и обновляется маркировка сети.

Лабораторная работа оформлена как воспроизводимый проект: исходные сценарии одновременно являются исполняемым кодом и literate-документацией. Такой подход соответствует идее литературного программирования [3] и реализован через `Literate.jl` [4]. Структура проекта организована с помощью `DrWatson.jl` [5], вычисления выполнены на Julia [6].

4 Выполнение лабораторной работы

4.1 Подготовка окружения

Работа выполнялась в каталоге проекта `~/labs/lab05/project`. На Рисунок 4.1 показан переход в рабочий каталог.

```
lilidji@eiglushchenko:~$ cd /home/lilidji/labs/lab05/project
lilidji@eiglushchenko:~/labs/lab05/project$ ^C
```

Рисунок 4.1: Переход в каталог проекта лабораторной работы

Версия Julia была проверена отдельной командой (Рисунок 4.2). Это подтверждает наличие рабочей среды для выполнения скриптов.

```
lilidji@eiglushchenko:~/labs/lab05/project$ ~/.juliaup/bin/julia --version
julia version 1.12.5
```

Рисунок 4.2: Проверка версии Julia

Далее было активировано окружение проекта и выполнена предкомпиляция зависимостей (Рисунок 4.3). В проекте использованы пакеты `CSV`, `DataFrames`, `DrWatson`, `CairoMakie`, `Literate`, `IJulia` и `ImageIO`.

```
lilidji@eiglushchenko:~/labs/lab05/project$ ~/.juliaup/bin/julia --project=. -e 'using Pkg; Pkg.instantiate(); Pkg.precompile()'
```

Рисунок 4.3: Подготовка окружения Julia-проекта

4.2 Структура проекта

Основные файлы лабораторной работы:

- `src/DiningPhilosophers.jl` – модуль с описанием структуры сети Петри, моделирования, проверки deadlock и построения графиков;
- `scripts/dining_philosophers.jl` – базовый эксперимент;
- `scripts/dining_philosophers_animation.jl` – построение GIF-анимации;
- `scripts/dining_philosophers_report.jl` – итоговый сравнительный график;
- `scripts/dining_philosophers_params.jl` – параметрическое исследование;
- `scripts/tangle.jl` – генерация clean-скриптов, Quarto-документов и notebook-файлов.

4.3 Описание программной модели

В модуле `DiningPhilosophers.jl` определена структура:

```julia
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end
```

Матрица инцидентности хранит изменение маркировки при срабатывании каждого перехода. Отрицательные элементы соответству-

ют удалению фишек из входных позиций, положительные – добавлению фишек в выходные позиции.

Функция `build_classical_network(N)` строит классическую сеть Петри для `N` философов. Функция `build_arbiter_network(N)` строит модифицированную сеть, дополнительно вводя позицию `Arbiter`. Проверка взаимной блокировки выполняется функцией `detect_deadlock(df, net)`: она анализирует последнюю маркировку и возвращает `true`, если ни один переход больше не может сработать.

4.4 Базовый эксперимент

Базовый эксперимент был запущен командой `scripts/dining_philosophers.jl` (Рисунок 4.4). Для `N = 5`, `tmax = 50.0` и `seed = 123` были смоделированы две сети: классическая и сеть с арбитром.

Рисунок 4.4: Запуск базового эксперимента

Для классической сети финальное время составило `6.764`, число событий – `20`, итоговое число голодных философов – `5`, а число философов в состоянии `Eat` – `0`. Функция обнаружения взаимной блокировки вернула `deadlock: true`.

Для сети с арбитром моделирование дошло до `t = 50.0`, число событий составило `105`, в финальном состоянии один философ находился в состоянии `Eat`, а признак блокировки равен `deadlock: false`.

CSV-файлы, созданные базовым экспериментом:

- `data/dining_classic.csv`;
- `data/dining_arbiter.csv`.

Графики, созданные базовым экспериментом:

- `plots/classic_simulation.png`;
- `plots/arbiter_simulation.png`.

4.5 Результаты классической сети

На Рисунок 4.5 показана динамика маркировки для классической сети. Верхние панели отражают состояния `Think_i` и `Hungry_i`, нижние – `Eat_i` и `Fork_i`.

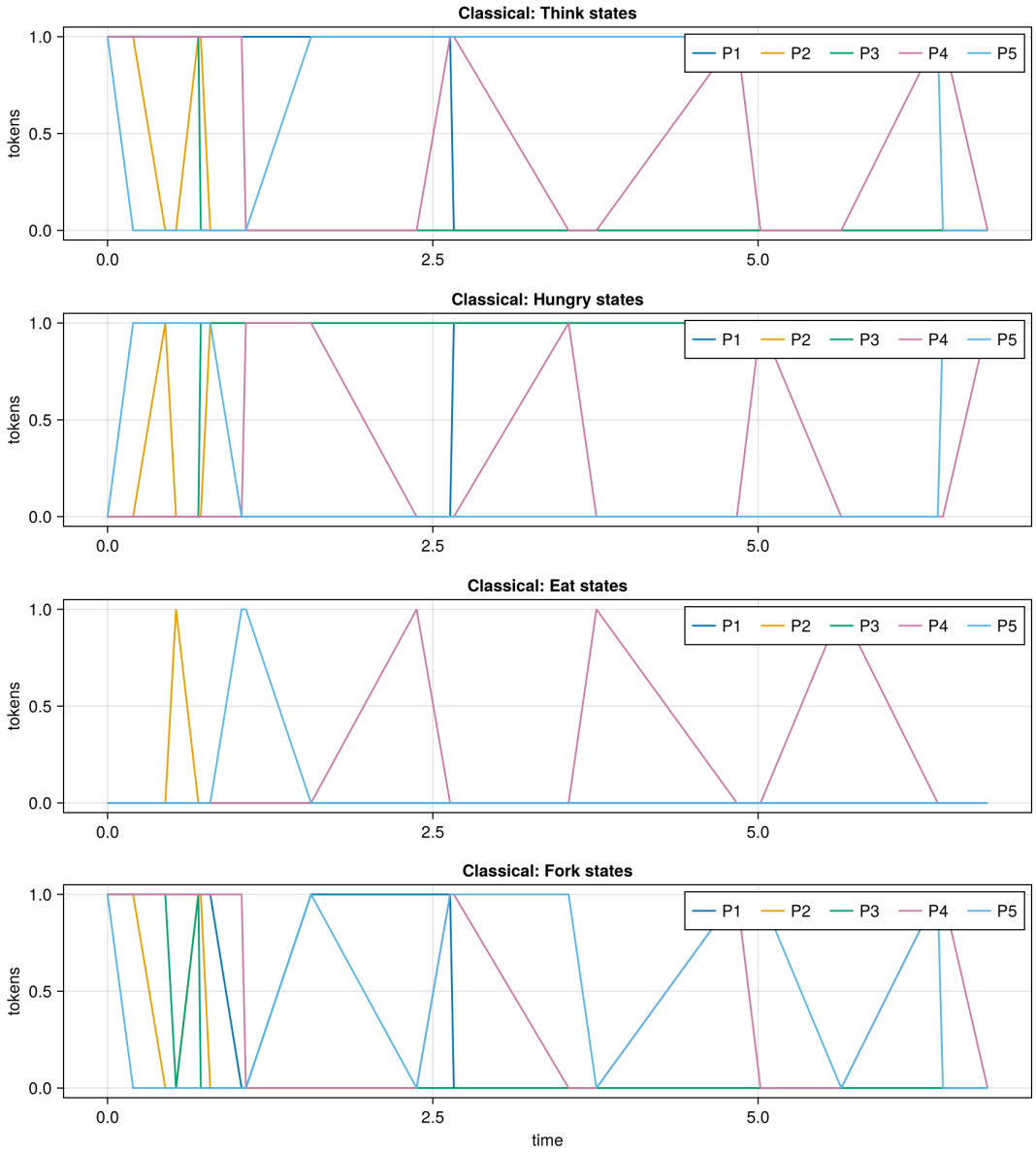

Рисунок 4.5: Эволюция маркировки классической сети

В конце моделирования все философы находятся в состояниях `Hungry_i`, все `Eat_i = 0`, все `Fork_i = 0`. Это означает, что каждый философ ожидает вторую вилку, но ни одна вилка недоступна. Полученная конфигурация соответствует deadlock.

Последняя строка `dining_classic.csv` имеет смысл:

- `time = 6.764274;`
- `Hungry_1 = ... = Hungry_5 = 1;`
- `Eat_1 = ... = Eat_5 = 0;`
- `Fork_1 = ... = Fork_5 = 0.`

4.6 Результаты сети с арбитром

На Рисунок 4.6 показана динамика маркировки для сети с арбитром.

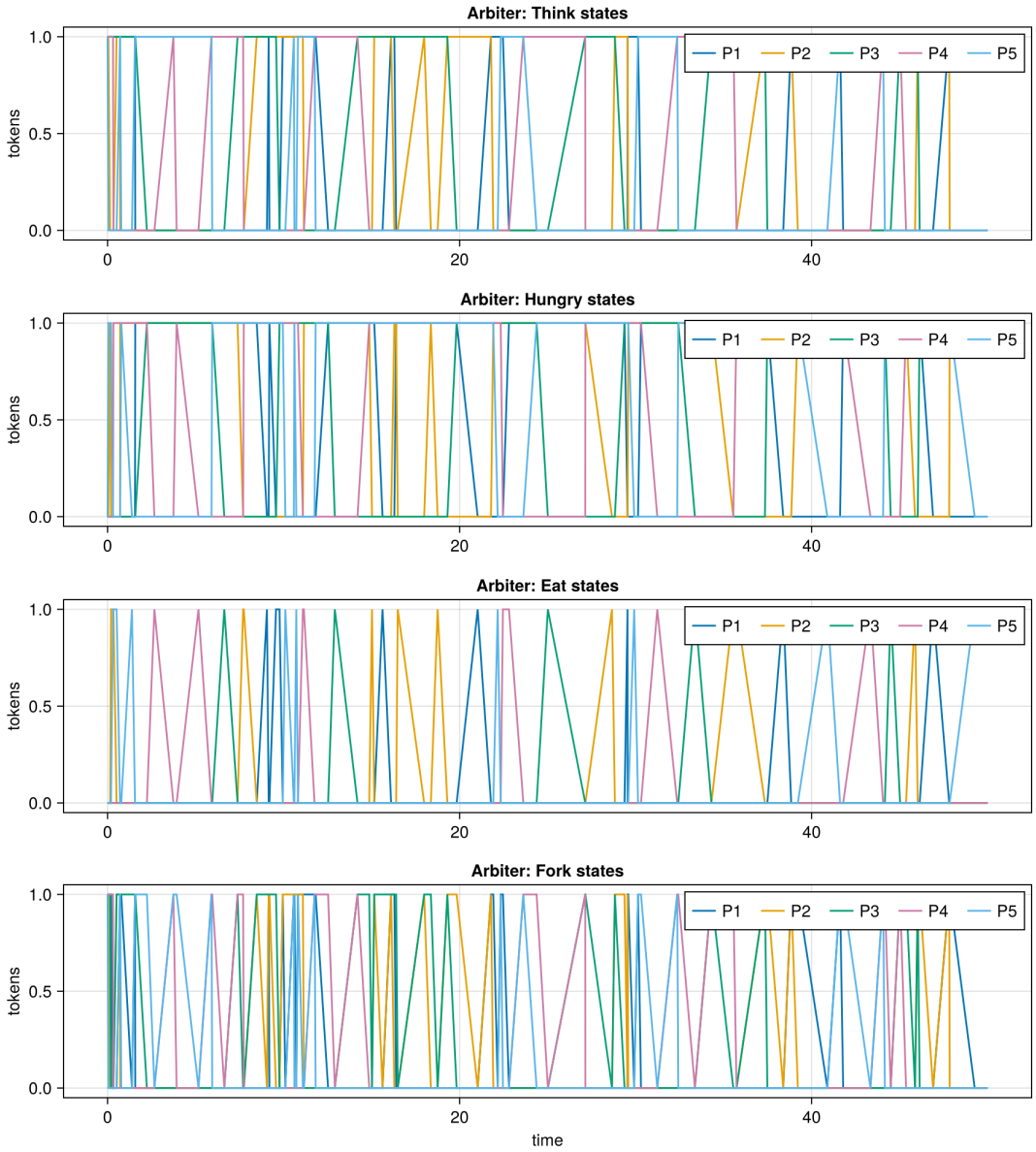

Рисунок 4.6: Эволюция маркировки сети с арбитром

В отличие от классической сети, переходы продолжают срабатывать до конца интервала моделирования. Позиция `Arbiter` ограничивает число философов, которые одновременно начинают захват вилок. Поэтому система не попадает в конфигурацию, где все философы удерживают по одной вилке.

4.7 Анимация процесса

Для наглядного просмотра изменения маркировки был запущен скрипт `scripts/dining_philosophers_animation.jl` (Рисунок 4.7). В нём используется классическая сеть с `N = 3`, чтобы подписи позиций оставались читаемыми.

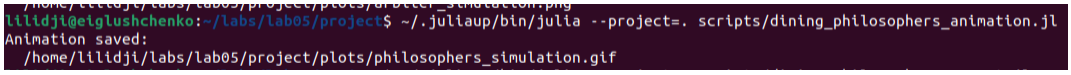

Рисунок 4.7: Запуск сценария построения анимации

Результатом стал файл `plots/philosophers_simulation.gif`. Анимация показывает последовательное перемещение фишек между позициями `Think`, `Hungry`, `Eat` и `Fork`.

4.8 Итоговый сравнительный график

Скрипт `scripts/dining_philosophers_report.jl` загружает CSV-файлы базового эксперимента и строит сравнительный график по состояниям `Eat_i` (Рисунок 4.8).

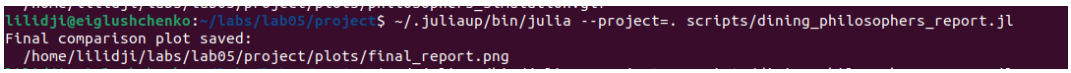

Рисунок 4.8: Запуск сценария итогового сравнительного графика

Итоговый график приведён на Рисунок 4.9. Верхняя панель соответствует классической сети, нижняя – сети с арбитром.

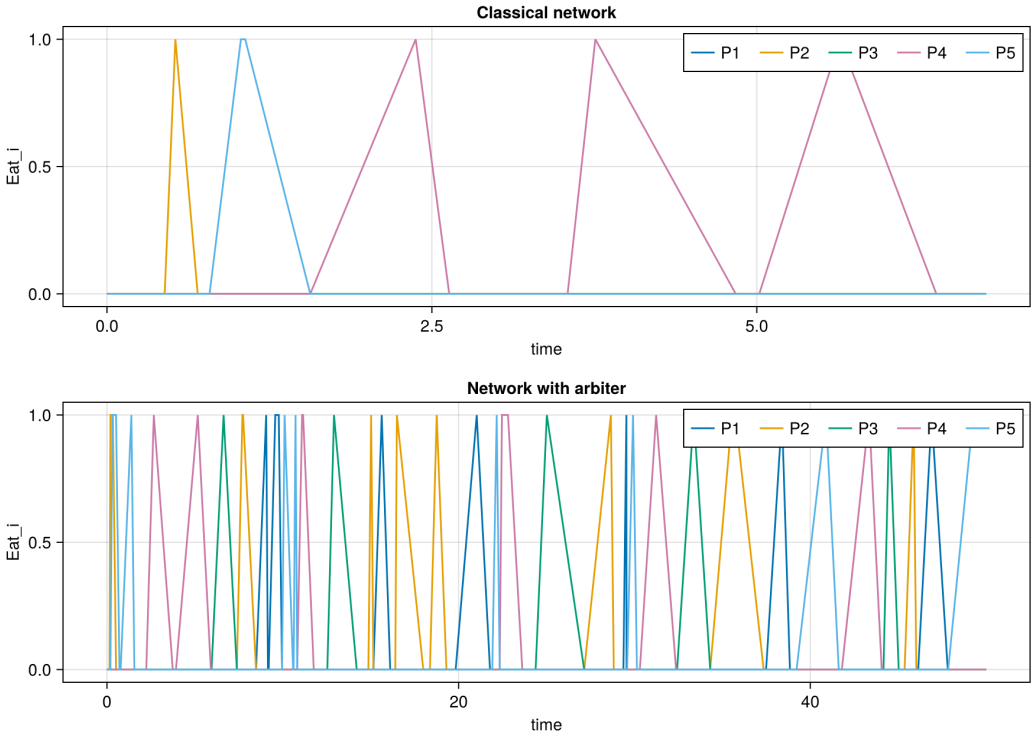

Рисунок 4.9: Сравнение состояний `Eat_i` для двух сетей

В классической сети линии `Eat_i` быстро опускаются к нулю и далее остаются нулевыми. В сети с арбитром состояния `Eat_i` продолжают появляться на всём интервале моделирования, что подтверждает отсутствие тупика.

4.9 Параметрическое исследование

Параметрическое исследование было выполнено скриптом `scripts/dining_philosophers_params.jl` (Рисунок 4.10). Использовались значения:

- `N = 3, 5, 7;`
- `tmax = 30.0, 50.0, 80.0;`
- `seed = 123, 124, 125.`

```
lilidji@elglushchenko:~/labs/lab05/project$ ~/.juliaup/bin/julia --project=. scripts/dining_philosophers_params.jl
Parameter study saved:
  /home/lilidji/labs/lab05/project/data/dining_params.csv
  /home/lilidji/labs/lab05/project/plots/dining_params.png

2×2 DataFrame
 Row │ network  deadlock_rate
     │ String   Float64
─────┼──────────────────────
   1 │ classic           1.0
   2 │ arbiter           0.0
```

Рисунок 4.10: Запуск параметрического исследования

Для каждой комбинации параметров моделировались две сети: `classic` и `arbiter`. Всего было выполнено 54 запуска. Результаты сохранены в файл `data/dining_params.csv`, а график частоты deadlock сохранён в `plots/dining_params.png`.

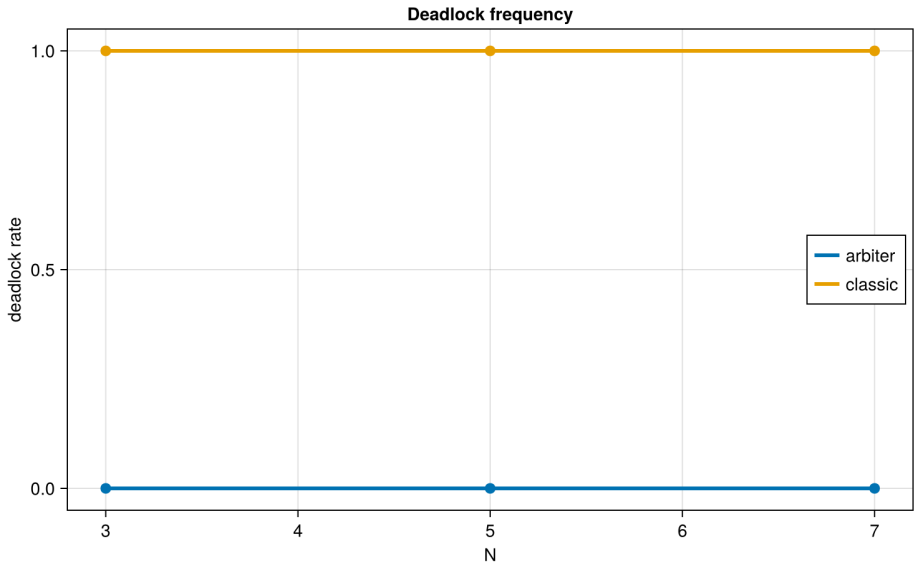

Рисунок 4.11: Частота возникновения deadlock в параметрическом исследовании

Полученный результат устойчив:

- для сети `classic` частота deadlock равна `1.0`;
- для сети `arbiter` частота deadlock равна `0.0`.

Следовательно, при рассмотренных значениях `N`, `tmax` и `seed` классическая сеть всегда приходила к взаимной блокировке, а сеть с арбитром не блокировалась ни в одном прогоне.

4.10 Генерация literate-форматов

После выполнения вычислительных сценариев был запущен `scripts/tangle.jl` (Рисунок 4.12). Он сформировал для каждого literate-скрипта три производных представления:

- clean-скрипт `.jl`;
- `Jupyter notebook` `.ipynb`;
- Quarto-документ `.qmd`.

```
lilidji@eiglushchenko:~/labs/lab05/project$ ~/.juliaup/bin/julia --project=. scripts/tangle.jl scripts/dining_philosophers.jl
[ Info: generating plain script file from `~/labs/lab05/project/scripts/dining_philosophers.jl`
[ Info: writing result to `~/labs/lab05/project/scripts/dining_philosophers/dining_philosophers.jl`
[ Info: generating notebook from `~/labs/lab05/project/scripts/dining_philosophers.jl`
[ Info: writing result to `~/labs/lab05/project/notebooks/dining_philosophers/dining_philosophers.ipynb`
[ Info: generating markdown page from `~/labs/lab05/project/scripts/dining_philosophers.jl`
[ Info: writing result to `~/labs/lab05/project/docs/dining_philosophers/dining_philosophers.qmd`
generated for dining_philosophers
  script   -> /home/lilidji/labs/lab05/project/scripts/dining_philosophers/dining_philosophers.jl
  notebook -> /home/lilidji/labs/lab05/project/notebooks/dining_philosophers/dining_philosophers.ipynb
  quarto   -> /home/lilidji/labs/lab05/project/docs/dining_philosophers/dining_philosophers.qmd
lilidji@eiglushchenko:~/labs/lab05/project$ ~/.juliaup/bin/julia --project=. scripts/tangle.jl scripts/dining_philosophers_animation.jl
[ Info: generating plain script file from `~/labs/lab05/project/scripts/dining_philosophers_animation.jl`
[ Info: writing result to `~/labs/lab05/project/scripts/dining_philosophers_animation/dining_philosophers_animation.jl`
[ Info: generating notebook from `~/labs/lab05/project/scripts/dining_philosophers_animation.jl`
[ Info: writing result to `~/labs/lab05/project/notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb`
[ Info: generating markdown page from `~/labs/lab05/project/scripts/dining_philosophers_animation.jl`
[ Info: writing result to `~/labs/lab05/project/docs/dining_philosophers_animation/dining_philosophers_animation.qmd`
generated for dining_philosophers_animation
  script   -> /home/lilidji/labs/lab05/project/scripts/dining_philosophers_animation/dining_philosophers_animation.jl
  notebook -> /home/lilidji/labs/lab05/project/notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
  quarto   -> /home/lilidji/labs/lab05/project/docs/dining_philosophers_animation/dining_philosophers_animation.qmd
lilidji@eiglushchenko:~/labs/lab05/project$ ~/.juliaup/bin/julia --project=. scripts/tangle.jl scripts/dining_philosophers_report.jl
[ Info: generating plain script file from `~/labs/lab05/project/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/labs/lab05/project/scripts/dining_philosophers_report/dining_philosophers_report.jl`
[ Info: generating notebook from `~/labs/lab05/project/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/labs/lab05/project/notebooks/dining_philosophers_report/dining_philosophers_report.ipynb`
[ Info: generating markdown page from `~/labs/lab05/project/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/labs/lab05/project/docs/dining_philosophers_report/dining_philosophers_report.qmd`
generated for dining_philosophers_report
  script   -> /home/lilidji/labs/lab05/project/scripts/dining_philosophers_report/dining_philosophers_report.jl
  notebook -> /home/lilidji/labs/lab05/project/notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
  quarto   -> /home/lilidji/labs/lab05/project/docs/dining_philosophers_report/dining_philosophers_report.qmd
lilidji@eiglushchenko:~/labs/lab05/project$ ~/.juliaup/bin/julia --project=. scripts/tangle.jl scripts/dining_philosophers_params.jl
[ Info: generating plain script file from `~/labs/lab05/project/scripts/dining_philosophers_params.jl`
[ Info: writing result to `~/labs/lab05/project/scripts/dining_philosophers_params/dining_philosophers_params.jl`
[ Info: generating notebook from `~/labs/lab05/project/scripts/dining_philosophers_params.jl`
[ Info: writing result to `~/labs/lab05/project/notebooks/dining_philosophers_params/dining_philosophers_params.ipynb`
[ Info: generating markdown page from `~/labs/lab05/project/scripts/dining_philosophers_params.jl`
[ Info: writing result to `~/labs/lab05/project/docs/dining_philosophers_params/dining_philosophers_params.qmd`
generated for dining_philosophers_params
  script   -> /home/lilidji/labs/lab05/project/scripts/dining_philosophers_params/dining_philosophers_params.jl
  notebook -> /home/lilidji/labs/lab05/project/notebooks/dining_philosophers_params/dining_philosophers_params.ipynb
  quarto   -> /home/lilidji/labs/lab05/project/docs/dining_philosophers_params/dining_philosophers_params.qmd
```

Рисунок 4.12: Генерация clean, notebook и Quarto-версий скриптов

Итоговая структура файлов показана на Рисунок 4.13. В каталоге `plots` находятся графики и GIF-анимация, в `data` – CSV-файлы, в `scripts/<name>` – clean-версии, в `notebooks/<name>` – notebook-файлы, в `docs/<name>` – Quarto-документы.

```
lilidji@eiglushchenko:~/labs/lab05/project$ ls -l plots | sort
find data -maxdepth 1 -type f | sort
find scripts -maxdepth 2 -type f | sort
find notebooks -maxdepth 2 -type f | sort
find docs -maxdepth 2 -type f | sort
arbiter_simulation.png
classic_simulation.png
dining_params.png
final_report.png
philosophers_simulation.gif
data/dining_arbiter.csv
data/dining_classic.csv
data/dining_params.csv
scripts/dining_philosophers_animation/dining_philosophers_animation.jl
scripts/dining_philosophers_animation.jl
scripts/dining_philosophers/dining_philosophers.jl
scripts/dining_philosophers.jl
scripts/dining_philosophers_params/dining_philosophers_params.jl
scripts/dining_philosophers_params.jl
scripts/dining_philosophers_report/dining_philosophers_report.jl
scripts/dining_philosophers_report.jl
scripts/tangle.jl
notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
notebooks/dining_philosophers/dining_philosophers.ipynb
notebooks/dining_philosophers_params/dining_philosophers_params.ipynb
notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
docs/dining_philosophers_animation/dining_philosophers_animation.qmd
docs/dining_philosophers/dining_philosophers.qmd
docs/dining_philosophers_params/dining_philosophers_params.qmd
docs/dining_philosophers_report/dining_philosophers_report.qmd
lilidji@eiglushchenko:~/labs/lab05/project$
```

Рисунок 4.13: Итоговая структура созданных файлов

5 Анализ результатов

Полученные результаты согласуются между собой:

- `dining_classic.csv` фиксирует переход классической сети в тупиковую маркировку;
- `classic_simulation.png` визуально показывает исчезновение свободных вилок и состояний `Eat_i`;
- `dining_arbiter.csv` показывает продолжение срабатывания переходов до конца интервала моделирования;
- `arbiter_simulation.png` демонстрирует чередование состояний в сети с арбитром;
- `final_report.png` показывает ключевое отличие двух сетей по состояниям `Eat_i`;
- `dining_params.csv` и `dining_params.png` подтверждают устойчивость результата на серии параметров.

Основной вывод состоит в том, что классическая постановка задачи обедающих философов содержит возможность deadlock, а введение дополнительного ресурса `Arbiter` предотвращает эту ситуацию.

6 Выводы

В ходе лабораторной работы был изучен аппарат сетей Петри и реализована модель задачи обедающих философов. Были построены две сети: классическая сеть и сеть с арбитром. Классическая сеть при `N = 5` пришла к взаимной блокировке за `20` событий и остановилась в момент времени `6.764`. Сеть с арбитром продолжила работу до конца интервала `tmax = 50.0` и не перешла в deadlock.

Были получены CSV-файлы с траекториями состояний, графики эволюции маркировки, GIF-анимация, итоговый сравнительный график и результаты параметрического исследования. Для всех 27 комбинаций параметров классическая сеть завершалась deadlock, а сеть с арбитром не блокировалась ни в одном запуске.

Также были подготовлены literate-версии скриптов и автоматически сгенерированы производные форматы `.jl`, `.qmd` и `.ipynb`, что обеспечивает воспроизводимость лабораторной работы.

1. Peterson J. L. Petri Net Theory and the Modeling of Systems. Englewood Cliffs, NJ: Prentice Hall, 1981.
2. Gillespie D. T. Exact Stochastic Simulation of Coupled Chemical Reactions // The Journal of Physical Chemistry. 1977. Т. 81, № 25. С. 2340–2361.
3. Knuth D. E. Literate Programming // The Computer Journal. 1984. Т. 27, № 2. С. 97–111.

4. Ekre F., contributors. Literate.jl Documentation. 2024.
5. Datseris G. и др. DrWatson.jl Documentation. 2024.
6. Bezanson J. и др. Julia: A Fresh Approach to Numerical Computing. 2017.